

NATURAL LANGUAGE PROCESSING

UNIT-II

Grammars and Parsing Grammars and Parsing – Top – Down and Bottom-Up Parsers, Transition Network Grammars, Feature Systems and Augmented Grammars, Morphological Analysis and the Lexicon, Parsing with Features, Augmented Transition Networks, Bayes Rule, Shannon game, Entropy and Cross Entropy

Grammars and Parsing in NLP

Introduction to Grammars in NLP

In Natural Language Processing (NLP), a **grammar** is a formal system that defines the rules governing how words are combined to form valid sentences in a language. Grammar provides a structured way for computers to understand the syntax of human language. Without grammar, a machine would only see a sentence as a sequence of unrelated words. By using grammatical rules, NLP systems can identify sentence structure, phrase boundaries, and relationships between words.

Grammars are essential because they enable machines to distinguish between grammatically correct and incorrect sentences. For example, the sentence *"The boy eats an apple"* follows English grammatical rules, while *"Boy the apple eats"* violates the standard word order. Grammar helps NLP systems detect such violations and interpret meaning correctly.

What is Parsing?

Parsing is the process of analyzing a sentence according to a given grammar to determine its syntactic structure. In NLP, parsing transforms a plain sentence into a structured representation, usually in the form of a **parse tree**. This tree shows how words group together into phrases and how these phrases relate to each other.

Parsing plays a crucial role in understanding natural language. It helps identify the subject, verb, and object of a sentence, which is necessary for tasks like machine translation, question answering, and information extraction.

Example:

Sentence: *The boy eats an apple*

Parsing identifies:

- Subject (NP): The boy
- Verb (V): eats
- Object (NP): an apple

This structural understanding allows an NLP system to correctly interpret who is performing the action and what the action affects.

Context-Free Grammar (CFG)

A **Context-Free Grammar (CFG)** is the most widely used grammar formalism in NLP. It is called “context-free” because each production rule can be applied regardless of the surrounding symbols. CFGs are powerful enough to describe the syntax of most natural language constructs while remaining computationally manageable.

A CFG consists of four components:

- **Non-terminals:** Abstract symbols representing syntactic categories (e.g., S, NP, VP).
- **Terminals:** Actual words of the language (e.g., boy, eats, apple).
- **Production rules:** Rules that describe how symbols can be replaced.
- **Start symbol:** The root symbol, usually S (Sentence).

Example CFG:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V NP$

$Det \rightarrow the \mid an$

$N \rightarrow boy \mid apple$

$V \rightarrow eats$

Using these rules, the sentence *"The boy eats an apple"* can be generated and analyzed.

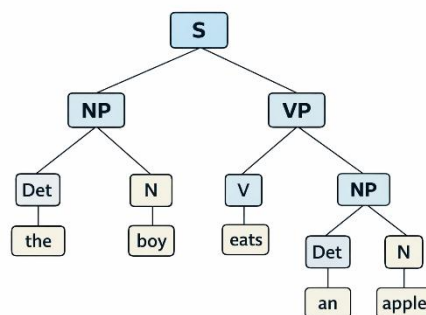
Parse Tree

A **parse tree** is a hierarchical representation of the syntactic structure of a sentence derived using a grammar. The root of the tree is the start symbol, internal nodes are non-terminals, and leaf nodes are terminal words.

Parse trees help visualize how a sentence is generated from grammar rules. They clearly show phrase boundaries such as noun phrases (NP) and verb phrases (VP).

Example Parse Tree:

Sentence: *The boy eats an apple*



This tree shows how the sentence is broken into meaningful grammatical components.

Parsing Strategies in NLP

Parsing strategies define how a parser applies grammar rules to analyze a sentence. The two fundamental strategies used in NLP are **Top-Down Parsing** and **Bottom-Up Parsing**. These strategies differ mainly in the direction of analysis and the order in which grammar rules are applied.

Top-Down Parsing

Definition

Top-Down Parsing is a parsing approach that starts from the start symbol (S) and attempts to generate the input sentence by repeatedly applying grammar rules. The parser works from the highest-level structure down to the individual words.

Example of Top-Down Parsing

Grammar:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V NP$

$Det \rightarrow the \mid an$

$N \rightarrow boy \mid apple$

$V \rightarrow eats$

Sentence: *the boy eats an apple*

Step-by-step process:

1. Start with S
2. Expand $S \rightarrow NP VP$
3. Expand $NP \rightarrow Det N \rightarrow the\ boy$
4. Expand $VP \rightarrow V NP \rightarrow eats\ an\ apple$

The sentence is successfully generated from the start symbol.

Characteristics

Top-down parsing predicts sentence structure before reading all words. It is simple but may attempt invalid rule expansions.

Bottom-Up Parsing

Definition

Bottom-Up Parsing starts from the input words and gradually combines them into larger grammatical units until the start symbol is reached.

Example of Bottom-Up Parsing

Sentence: *the boy eats an apple*

Steps:

1. Identify words: the | boy | eats | an | apple

2. Combine Det + N $\rightarrow$ NP $\rightarrow$ the boy
3. Combine Det + N $\rightarrow$ NP $\rightarrow$ an apple
4. Identify V $\rightarrow$ eats
5. Combine V + NP $\rightarrow$ VP
6. Combine NP + VP $\rightarrow$ S

The sentence is successfully reduced to the start symbol.

Characteristics

Bottom-up parsing is data-driven and avoids left-recursion issues.

Comparison of Top-Down and Bottom-Up Parsing

Top-down parsing begins with the grammar and tries to fit the sentence, while bottom-up parsing begins with the sentence and builds toward the grammar. Top-down methods are simpler but less efficient, whereas bottom-up methods are more powerful and suitable for real-world NLP systems.

Role of Parsing in NLP Applications

Parsing is a core component of many NLP applications such as machine translation, grammar checking, question answering, chatbots, and information extraction. Accurate parsing improves language understanding and enables machines to process text more intelligently.

Top-Down Parsing:

Grammar

$S \rightarrow NP VP$

$NP \rightarrow ART N$

$NP \rightarrow ART ADJ N$

$VP \rightarrow V$

$VP \rightarrow V NP$

“the small child plays the game”

Word index:

1 the | 2 small | 3 child | 4 plays | 5 the | 6 game

Step	Current State	Backup State	Remaining Sentence	Explanation
1	(S, 1)	—	the small child plays the game	Start symbol
2	(NP VP, 1)	—	the small child plays the game	Apply S → NP VP
3	(ART N VP, 1)	—	the small child plays the game	Try NP → ART N
		(ART ADJ N VP, 1)	the small child plays the game	Backup rule
4	(N VP, 2)	(ART ADJ N VP, 1)	small child plays the game	Match ART = the
5	fail	(ART ADJ N VP, 1)	small child plays the game	N ≠ small
6	(ADJ N VP, 2)	—	small child plays the game	Backtrack
7	(N VP, 3)	—	child plays the game	Match ADJ = small
8	(VP, 4)	—	plays the game	Match N = child
9	(V, 4)	(V NP, 4)	plays the game	Try VP → V
10	fail	(V NP, 4)	the game	Input remains
11	(NP, 5)	—	the game	Backtrack → VP → V NP
12	(ART N, 5)	(ART ADJ N, 5)	the game	Expand NP
13	(N, 6)	(ART ADJ N, 5)	game	Match ART = the
14	(ε)	—	—	Match N = game , success

Bottom-up Parser:

A bottom-up parser constructs a parse tree from leaves to root by shifting input symbols and reducing them using grammar rules until the start symbol is obtained.

Grammar (Given)

1. $S \rightarrow NP VP$
2. $NP \rightarrow ART N$
3. $NP \rightarrow ART ADJ N$
4. $VP \rightarrow V$
5. $VP \rightarrow V NP$

Lexical assumptions:

- $ART \rightarrow the$
- $ADJ \rightarrow small$
- $N \rightarrow boy, apple$
- $V \rightarrow eats$

Sentence: the small boy eats the apple

Step	Stack	Input	Action
1	the	small boy eats the apple	SHIFT
2	ART	small boy eats the apple	REDUCE ($ART \rightarrow the$)
3	ART small	boy eats the apple	SHIFT
4	ART ADJ	boy eats the apple	REDUCE ($ADJ \rightarrow small$)
5	ART ADJ boy	eats the apple	SHIFT
6	ART ADJ N	eats the apple	REDUCE ($N \rightarrow boy$)
7	NP	eats the apple	REDUCE ($NP \rightarrow ART ADJ N$)
8	NP eats	the apple	SHIFT
9	NP V	the apple	REDUCE ($V \rightarrow eats$)
10	NP V the	apple	SHIFT
11	NP V ART	apple	REDUCE ($ART \rightarrow the$)

Step	Stack	Input	Action
12	NP V ART apple	—	SHIFT
13	NP V ART N	—	REDUCE (N → apple)
14	NP V NP	—	REDUCE (NP → ART N)
15	NP VP	—	REDUCE (VP → V NP)
16	S	—	REDUCE (S → NP VP)

Transition Network Grammars :

A **Transition Network Grammar** is a type of **graph-based representation of a grammar** used in natural language processing (NLP) for parsing sentences.

It's called a *transition network* because it represents the flow or transitions between grammatical states (nodes) through arcs (edges) labeled with word categories or terminal words.

- TNGs model grammar as a **finite-state machine (FSM)**.
- Each **node** represents a **syntactic state**.
- Each **arc** represents a **transition** based on a word or a word category (like noun, verb, determiner).
- A **sentence is accepted** if a path exists from the **start node** to an **end node** that matches the sequence of words in the sentence.

Components of a Transition Network Grammar

1. Nodes (States):

Represent syntactic categories or points in parsing, e.g., S (sentence), NP (noun phrase), VP (verb phrase).

2. Arcs (Edges):

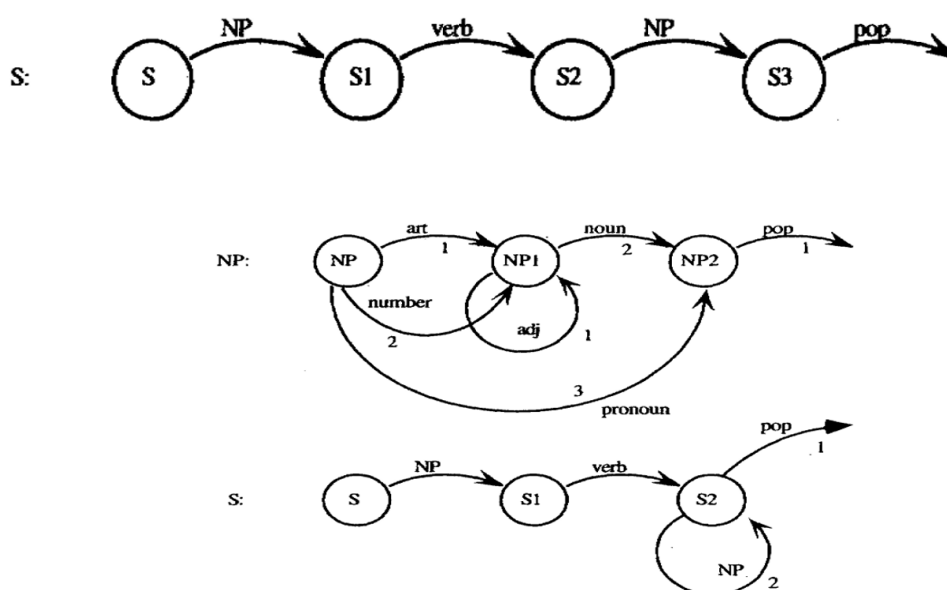
Labeled with:

1. **Terminal symbols:** Specific words like "runs", "dog", "John".
2. **Non-terminal symbols:** Higher-level categories like NP, VP.
3. **Start Node:**
The node where parsing begins (usually S for a sentence).
4. **End Node:**
Accepting state that indicates the sentence has been successfully parsed.

Procedure:

1. Start at the initial node.
2. For each word in the sentence, check if there's an arc that matches the current word or its category.
3. Follow the arc to the next node.
4. Repeat until all words are consumed.
5. If the parser reaches the final node, the sentence is accepted. Otherwise, it's rejected.

NP -> ART NP1
NP1 -> ADJ NP1
NP1 -> N



Feature Systems and Augmented Grammars:

Why Feature Systems are Needed

In Natural language, **words and phrases must agree with each other**. Simple Context-Free Grammars (CFGs) cannot easily handle these agreement constraints.

Examples of agreement

- **Number agreement:**
 - ✗ *a men* (incorrect)
 - ✓ *a man, the men*
- **Subject-verb agreement:**
 - ✗ *She eat rice*
 - ✓ *She eats rice*
- **Gender agreement:**
 - ✗ *He lost her book* (when referring to a male)
- **Head-complement restrictions:**

Certain verbs allow only specific complements.

To handle such constraints efficiently, grammars are extended using **Feature Systems** and **Augmented Grammars**.

Limitation of Context-Free Grammars

If agreement constraints are handled only using CFGs, separate rules must be written for each form.

```
NP-SING → ART-SING N-SING  
NP-PLURAL → ART-PLURAL N-PLURAL
```

This approach increases the size of the grammar. If more features such as person or gender are added, the number of rules grows rapidly.

Hence, CFG-based solutions are inefficient and difficult to maintain.

Feature Systems:

- A **Feature system** allows grammatical constituents to carry additional information called **features**, such as NUMBER, PERSON, and GENDER.
- Each feature has a specific value,
for example **NUMBER = singular (s) or plural (p)**.
- By using features, a single grammar rule can handle multiple agreement conditions without duplicating rules.
- **Augmented Grammars:**
- In **Natural Language Processing (NLP)**, an **Augmented Grammar** is a grammar that extends ordinary grammar rules by adding **extra information such as actions, conditions, or feature checks**.
- These additions help the system not only to analyze the structure of a sentence but also to verify agreement, meaning, and constraints during parsing.
- **Augmented Grammars:**
- An **augmented grammar** is a context-free grammar extended with features and agreement conditions.

For example, the rule:

NP → ART N (only when NUMBER of ART agrees with NUMBER of N)

states that a noun phrase is valid only if both the article and noun have the same NUMBER feature. This single rule replaces multiple CFG rules and enforces agreement naturally.

Feature Structures

In augmented grammars, each constituent is represented as a **feature structure**, which is a collection of feature–value pairs.

For example, the article “a” can be represented as:

(ART ROOT a NUMBER s)

This indicates that “*a*” is an article whose root is *a* and whose number is singular. Feature structures provide a precise way to represent grammatical information.

Feature Structures for Phrases

Feature structures can also represent larger phrases by embedding sub-constituents.

For example, the noun phrase “*a fish*” can be represented as:

```
(NP NUMBER s
  1 (ART ROOT a NUMBER s)
  2 (N ROOT fish NUMBER s))
```

Here, the noun phrase and both of its sub-constituents agree in number, making it a valid NP. This structure also corresponds to a parse tree.

Problem Description

In English, pronouns must agree in **gender** with the noun they refer to.

For example, the sentence “*John lost his book*” is correct, while “*John lost her book*” is incorrect because *John* is masculine and the pronoun *her* is feminine. This shows the need for **gender agreement** in natural language.

Introducing Gender as a Feature

To handle this problem, we introduce a **GENDER** feature with values such as *m* and *f*. Examples:

- *John* → GENDER = *m*
- *Mary* → GENDER = *f*
- *his* → GENDER = *m*
- *her* → GENDER = *f*

Feature Structures

The feature structure for the noun “*John*” can be written as:

```
(NP ROOT John GENDER m)
```

(PRON ROOT his GENDER m)

Augmented Grammar Rule

An augmented grammar rule enforcing gender agreement can be written as:

(NP GENDER ?g) → (PRON GENDER ?g) (N)

This rule states that a pronoun phrase is valid only if the pronoun and the noun agree in gender.

Morphological Analysis and the Lexicon

Lexical Morphology

Lexical morphology is a specialized area of morphology that focuses on **lexemes** in a language.

A **lexeme** is the fundamental unit of lexical meaning. It represents an abstract vocabulary item rather than a specific word form.

- A lexeme may appear as:
 - A **single word** (e.g., *run*)
 - A **set of related word forms** (run, runs, running, ran)

Lexical morphology studies how these lexemes behave and change when grammatical or derivational processes are applied.

Morphological Analysis

Morphological Analysis is the study of **lexemes and the processes by which they are formed**.

It examines how words are constructed from **root words** and how grammatical information is encoded.

This discipline is particularly interested in:

- **Neologisms** – newly created words derived from existing root words
- **Derivation** – forming new words using affixes
- **Compounding** – combining multiple words to form a new word

Neologisms

Newly created words formed from existing words or roots.

Examples:

- *Google* → *googling*
- *Blog* → *blogger*
- *Selfie*
- *Chat* → *chatbot*

Derivation

Forming new words by adding prefixes or suffixes to a root word.

Examples:

- *happy* → *unhappy*
- *teach* → *teacher*
- *nation* → *national*
- *care* → *careless*

Compounding

Combining two or more independent words to create a new word.

Examples:

- *black* + *board* → *blackboard*
- *rain* + *bow* → *rainbow*
- *tooth* + *brush* → *toothbrush*
- *class* + *room* → *classroom*

In morphological analysis, **each token (word occurrence)** is analyzed in the following format:

token → lemma (root word) + part of speech + grammatical features

Examples

- **cats** → cat + N + plural
- **played** → play + V + past

This representation helps NLP systems understand both the **structure and grammatical role** of a word.

Non-Determinism in Morphological Analysis

Morphological analysis is often **non-deterministic**, meaning a single word may have **more than one valid analysis**.

Example

Example 1: *unlockable*

This word has two valid morphological analyses:

Analysis 1

- *un + lock + able*

Meaning: not able to be locked

Analysis 2

- *unlock + able*

Meaning: able to be unlocked

Same word → different structure → different meaning.

Here, the correct interpretation depends on **context**, making ambiguity resolution an important task in NLP.

Derivation in Lexical Morphology

Derivation is a word-formation process that creates **new lexemes** by adding **affixes** to a root word.

This process is also called **affixation**.

Types of Affixes

There are two main types:

Prefixes (added before the root)

- rewrite → root: write
- unfair → root: fair

Suffixes (added after the root)

- wanted → root: want
- wants → root: want
- wanting → root: want

Derivation often changes:

- The **meaning** of a word
- Sometimes the **part of speech**

Compounding

Compounding refers to forming new words by **combining two or more existing words** into a single lexical item.

Examples

- green + house → greenhouse
- mother + in + law → mother-in-law
- motor + bike → motorbike
- cook + book → cookbook
- foot + ball → football

Compounds behave as **single lexemes** even though they consist of multiple words.

The Lexicon

The **lexicon** is a structured dictionary used by NLP systems.

It must contain information about **all words that can be used in the language**, along with their grammatical properties.

The lexicon includes:

- Root form (lemma)
- Part of speech
- Morphological features
- Feature value restrictions

If a word is **ambiguous**, the lexicon contains **multiple entries**, one for each possible interpretation.

<i>Word</i>	Part of Speech	Root	Morphological Features	Semantic Features
<i>cat</i>	Noun	cat	singular	animate, animal
<i>cats</i>	Noun	cat	plural	animate, animal
<i>teacher</i>	Noun	teach	derivation (-er)	human, agent
<i>run</i>	Verb	run	base form	action
<i>runs</i>	Verb	run	present, 3rd person, singular	action
<i>ran</i>	Verb	run	past tense	action
<i>running</i>	Verb	run	progressive	action
<i>happy</i>	Adjective	happy	positive degree	emotion
<i>unhappy</i>	Adjective	happy	prefix (un-)	negative
<i>happier</i>	Adjective	happy	comparative	emotion
<i>nation</i>	Noun	nation	base form	abstract
<i>nationalization</i>	Noun	nation	al + ize + ation	abstract process
<i>unlockable</i>	Adjective	lock	ambiguous morphology	ability/state

Verb Morphology in English

Most English verbs use a **common set of suffixes** to express grammatical forms:

- **-s** → third person singular present tense
- **-ed** → past tense
- **-ing** → present participle

Instead of storing every form separately, the lexicon stores the **base form** and uses **rules** to derive other forms.

Rule for Present Tense (Third Person Singular)

(V ROOT ?r SUBCAT ?s VFORM pres AGR 3s)

→

(V ROOT ?r SUBCAT ?s VFORM base) (+S)

Here:

- +S is a lexical category containing the suffix **-s**
- The rule derives the present tense form from the base verb

Example Lexicon Entry

want:

(V ROOT want

SUBCAT {_np_vp:inf _np_vp:inf}

VFORM base)

Given the input string **want + s**, the rule produces:

wants:

(V ROOT want

SUBCAT {_np_vp:inf _np_vp:inf}

VFORM pres

AGR 3s)

Present Tense (Non-Third Person Singular)

For most verbs, present tense forms for **non-third person** subjects are identical to the base form.

Rule:

(V ROOT ?r SUBCAT ?s VFORM pres AGR {1s 2s 1p 2p 3p})

→

(V ROOT ?r SUBCAT ?s VFORM base)

Handling Irregular Verbs

Some verbs have **irregular present tense forms**.

Example:

- Incorrect: *We be at the store*
- Correct: *We are at the store*

To avoid incorrect derivations, a binary feature **IRREG-PRES** is introduced.

Modified Rule

(V ROOT ?r SUBCAT ?s VFORM pres AGR ...)

→

(V ROOT ?r SUBCAT ?s VFORM base IRREG-PRES -)

This rule applies **only to regular verbs**, preventing erroneous interpretations.

Managing Feature Complexity

As the number of features increases, writing lexicon entries becomes complex.

To address this:

- Lexicons allow **feature clusters**
- A single symbol represents a group of features

This simplifies lexicon creation and maintenance.

Lexical Ambiguity: Example of “saw”

The word **saw** has multiple entries in the lexicon:

1. **Noun**
 - *The saw was broken.*
2. **Verb (base form)**
 - *Jack wanted me to saw the board in half.*
3. ***Verb (past tense of “see”)**

- *I saw Jack eat the pizza.*

Each usage has a separate lexicon entry.

Parsing with Features:

Parsing with features in NLP refers to analyzing sentence structure while attaching **linguistic features** to words and phrases so that grammatical constraints can be checked during parsing. Instead of relying only on phrase structure rules, the parser also verifies whether features such as number, person, tense, and gender are compatible.

Consider the sentence:

“This cat eats fish.”

Here, *cat* has the feature **number = singular** and *eats* also has **number = singular**. Because the features agree, the sentence is accepted by the parser.

Now look at:

“This cats eats fish.”

This expects a singular noun, while *cats* has **number = plural**. The mismatch in features causes the parse to fail. This shows how features help reject ungrammatical sentences.

Features are usually represented as **attribute–value pairs**.

For example:

- *cat* → {category: noun, number: singular}
- *cats* → {category: noun, number: plural}

During parsing, these feature structures must **unify**. Unification means that two structures can be combined only if their feature values do not conflict.

Feature-based grammar rules explicitly mention constraints.

For example, a noun phrase rule can be written as:

NP → Det N, with the condition that **NP.number = N.number**.

This ensures that phrases like **“a boy”** are accepted, while **“a boys”** is rejected.

Features are also useful for handling **verb agreement**.

In the sentence “**She runs fast**”,

- *she* → {person: third, number: singular}
- *runs* → {person: third, number: singular}

Since both agree, parsing succeeds.

But in “**She run fast**”,

- *run* → {person: first/second, number: plural}
- The mismatch causes a parsing error.

Parsing with features helps in resolving **ambiguity** as well.

Consider the sentence:

“**I saw the man with a telescope.**”

Features related to verb arguments and noun attachment can help decide whether *with a telescope* describes *how I saw* or *which man*.

Semantic features can also be included.

For example, in “**The stone ate food**”,

- *stone* → {animate: no}
- *eat* → {subject: animate}

Because the semantic features do not match, the parser can mark the sentence as semantically invalid.

In practical NLP applications like grammar checking, machine translation, and speech recognition, feature-based parsing improves accuracy by enforcing agreement and linguistic constraints. Although it is computationally heavier than simple context-free parsing, it produces more **linguistically correct and meaningful parses**.

In short, parsing with features enriches syntactic analysis by combining grammar rules with feature checking, making NLP systems more robust and closer to natural language understanding.

Augmented Transition Networks:

Augmented Transition Networks (ATNs) are a powerful parsing formalism in NLP that extend finite state machines by integrating **grammar rules, features, tests, and actions**. They are closely related to **context-free grammar (CFG)** but provide greater expressive power through augmentation.

Augmented Transition Networks (ATNs)

Augmented Transition Networks (ATNs) are an extension of Finite State Automata (FSA) used mainly in Natural Language Processing (NLP) for syntactic parsing of sentences.

An ATN is a **state-transition network** that is augmented with:

- **Registers (memory)**
- **Procedural tests**
- **Actions**

These additions make ATNs more powerful than simple FSAs and capable of representing **context-free grammars** and some **context-sensitive constraints**.

Why ATNs are Needed

Finite State Automata cannot handle:

- Nested structures (e.g., embedded clauses)
- Long-distance dependencies

ATNs overcome these limitations by allowing:

- **Recursion**
- **Feature checking**
- **Dynamic decision-making during parsing**

Components of an ATN

1. States (Nodes)

Represent positions in parsing.

2. Arcs (Transitions)

Types of arcs include:

- **Word arcs** → match specific words
- **Category arcs** → match grammatical categories (NP, VP, etc.)
- **Push arcs** → call a sub-network (recursion)
- **Pop arcs** → return from sub-network

3. Registers

Store information like:

- Number (singular/plural)
- Tense
- Case

4. Tests

Conditions that must be satisfied for a transition (e.g., subject–verb agreement).

5. Actions

Operations performed when a transition is taken (e.g., building parse trees).

Example

Sentence:

“The boys eat apples.”

ATN processing:

- NP network parses “*The boys*”
- VP network parses “*eat apples*”
- Register check ensures **plural subject** matches **plural verb**

Simple Grammar Representation Using ATN

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V NP$

In ATN:

- NP and VP are **separate subnetworks**
- The main S network calls NP and VP using **push arcs**

Advantages of ATNs

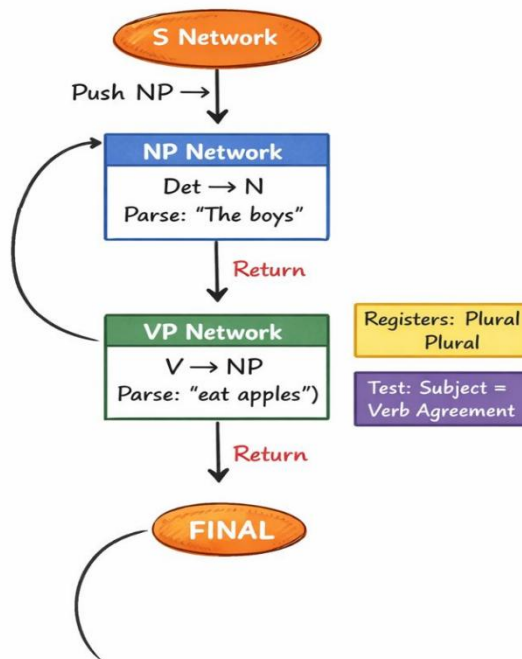
- ✓ Handles recursion and nested structures
- ✓ Supports agreement and constraints
- ✓ More expressive than FSAs

Limitations of ATNs

- ✗ Complex to design and maintain
- ✗ Parsing can be computationally expensive
- ✗ Largely replaced by probabilistic and neural parsers

Applications in NLP

- Syntactic parsing
- Grammar checking
- Early NLP systems (e.g., SHRDLU)
- Linguistic research



$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V NP$

$Det \rightarrow the$

$N \rightarrow boys \mid apples$

$V \rightarrow eat$

Explanation (Simple)

- **S (Sentence)** consists of a **Noun Phrase (NP)** and a **Verb Phrase (VP)**
- **NP** = Determiner + Noun
 - *the boys, apples*
- **VP** = Verb + Noun Phrase
 - *eat apples*

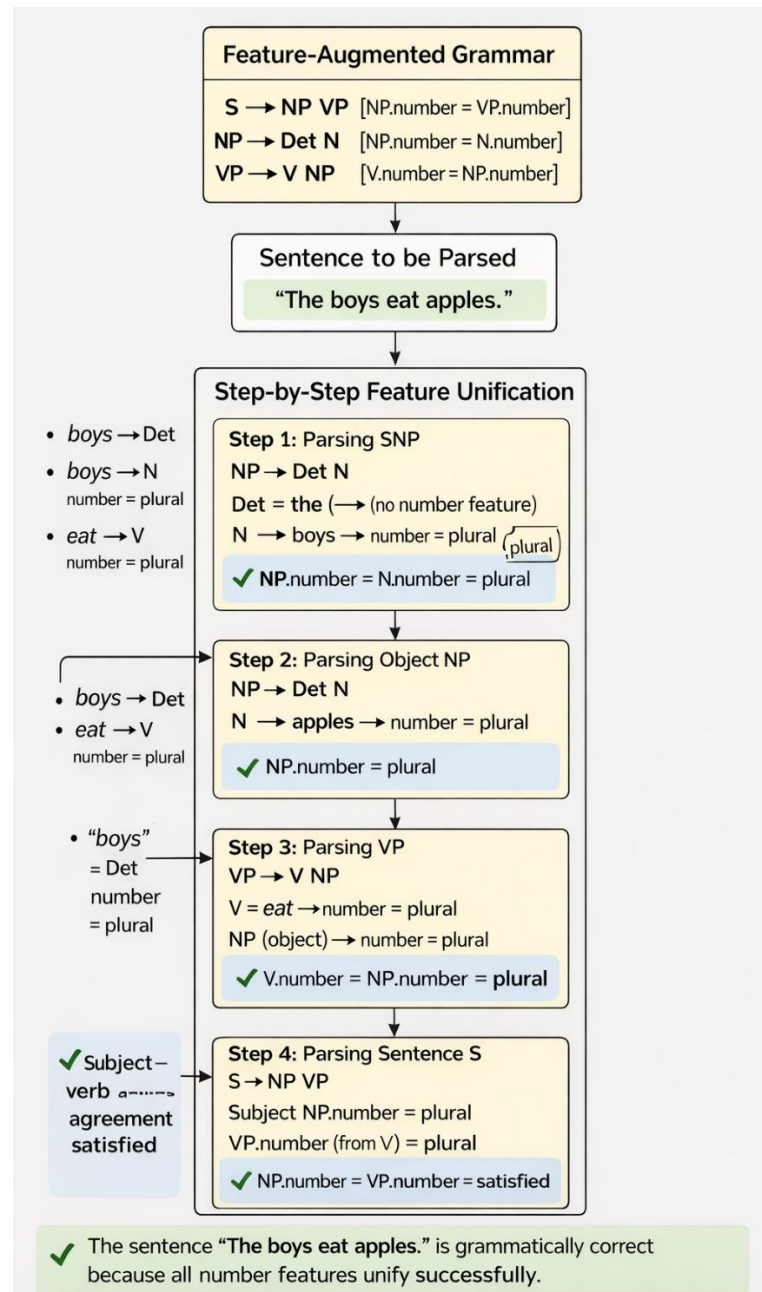
With Feature Agreement (Optional – for ATN)

$S \rightarrow NP VP$

[NP.number = VP.number]

NP → Det N
[NP.number = N.number]

VP → V NP
[V.number = NP.number]



The ATN stores **number = plural** in a register when parsing the subject NP. When parsing the verb, a test checks whether the verb agrees with the stored number. Since the features match, the parse succeeds.

Now consider the incorrect sentence:

“The boys eats apples.”

- *boys* → number = plural
- *eats* → number = singular

The ATN fails the agreement test on the VP arc, and the sentence is rejected.

ATNs can also represent **optional and alternative grammar rules**.

For example:

$VP \rightarrow V \ NP \mid V$

This allows both:

- **“She runs.”**
- **“She eats food.”**

In the ATN, this is modeled by two different paths leaving the same state, one that consumes only a verb and another that consumes a verb followed by an NP.

ATNs handle **embedded structures** using recursive grammar rules.

Consider the grammar:

$NP \rightarrow NP \ RelClause$

$RelClause \rightarrow who \ VP$

Sentence:

“The student who studies hard succeeds.”

The ATN for NP can call the RelClause network after completing the basic NP, parse *who studies hard*, and then return to the main sentence network. This closely mirrors the grammar structure.

In essence, **ATNs operationalize grammar rules** by turning them into executable networks. Grammar rules define the possible paths, features enforce constraints like agreement, and registers store intermediate information. This tight integration of grammar and parsing logic made ATNs a foundational technique in early rule-based NLP systems and an important bridge between formal grammar and practical parsing.

Bayees Rule:

Bayes' Rule is a fundamental principle in probability theory that explains **how to update our belief about an event when new evidence is observed**. It is widely used in NLP, machine learning, medical diagnosis, and decision-making systems.

Bayes' Rule is expressed as:

$$P(A | B) = (P(B | A) \times P(A)) / P(B)$$

Here,

- **P(A)** is the prior probability of event A
- **P(B | A)** is the likelihood of observing B given A
- **P(B)** is the probability of evidence B
- **P(A | B)** is the posterior probability, the updated belief about A after seeing B

Consider a **real-life example** related to email spam detection.

Let:

- A = Email is Spam
- B = Email contains the word "offer"

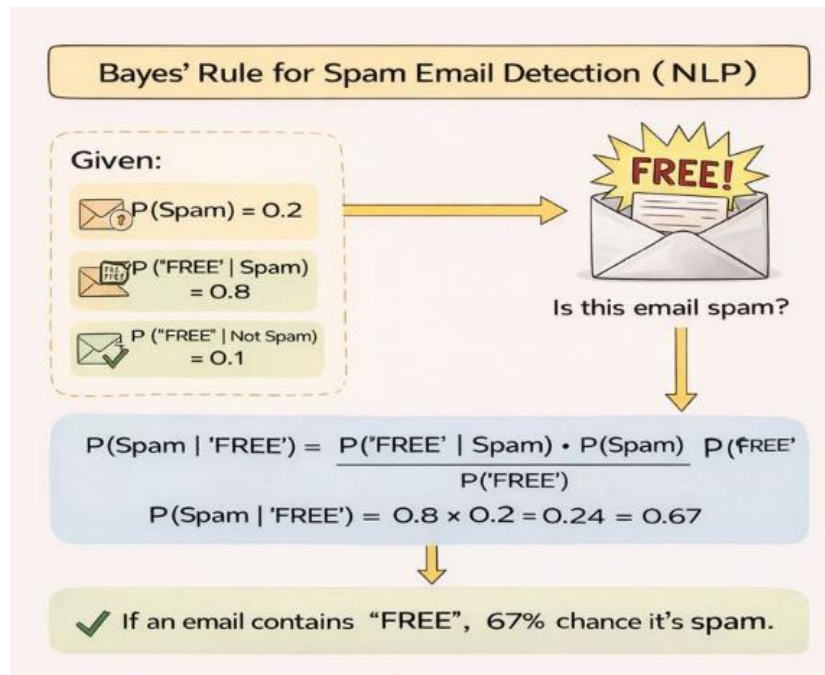
Suppose:

- $P(\text{Spam}) = 0.2$ (20% of emails are spam)
- $P(\text{"offer"} | \text{Spam}) = 0.8$
- $P(\text{"offer"}) = 0.3$

Using Bayes' Rule:

$$P(\text{Spam} | \text{"offer"}) = (0.8 \times 0.2) / 0.3 = 0.16 / 0.3 \approx 0.53$$

This means that **if an email contains the word "offer", there is a 53% chance that it is spam**, even though initially spam probability was only 20%. Bayes' Rule updates our belief using evidence.



Now consider a **medical diagnosis example**.

Let:

- A = Person has a disease
- B = Test result is positive

Assume:

- $P(\text{Disease}) = 0.01$
- $P(\text{Positive} | \text{Disease}) = 0.99$
- $P(\text{Positive}) = 0.05$

Then:

$$P(\text{Disease} | \text{Positive}) = (0.99 \times 0.01) / 0.05 = 0.0099 / 0.05 = 0.198$$

Even after a positive test, the probability of actually having the disease is only about **19.8%**. This example shows why Bayes' Rule is crucial in real-world decision-making.

In **Natural Language Processing**, Bayes' Rule is the foundation of the **Naïve Bayes classifier**.

For text classification, the goal is to find the most probable class given a document:

$$P(\text{Class} | \text{Document}) \propto P(\text{Document} | \text{Class}) \times P(\text{Class})$$

Example:

Sentence: “**Free prize offer**”

The classifier computes:

- Probability of the sentence given Spam
- Probability of the sentence given Not Spam

The class with the higher posterior probability is selected.

Bayes’ Rule also helps in **word sense disambiguation**.

Sentence:

“**He sat near the bank.**”

Let:

- A_1 = bank means riverbank
- A_2 = bank means financial institution

Words like *river* increase $P(A_1 \mid \text{context})$, while words like *money* increase $P(A_2 \mid \text{context})$.

Bayes’ Rule combines prior meaning frequency with contextual evidence.

Example

Problem:

A factory produces 2 types of bulbs:

- Type A: 60% of total ($P(A) = 0.6$)
- Type B: 40% of total ($P(B) = 0.4$)

Defective bulbs:

- Type A defective 1% ($P(D \mid A) = 0.01$)
- Type B defective 2% ($P(D \mid B) = 0.02$)

Question: If a randomly picked bulb is defective, what is the probability it is Type A?

Step 1: Use Bayes’ Rule

$$P(A | D) = \frac{P(D | A) \cdot P(A)}{P(D)}$$

Where $P(D)$ = total probability of defect:

$$P(D) = P(D | A)P(A) + P(D | B)P(B)$$

Step 2: Calculate $P(D)$

$$P(D) = (0.01 \cdot 0.6) + (0.02 \cdot 0.4)$$

$$P(D) = 0.006 + 0.008 = 0.014$$

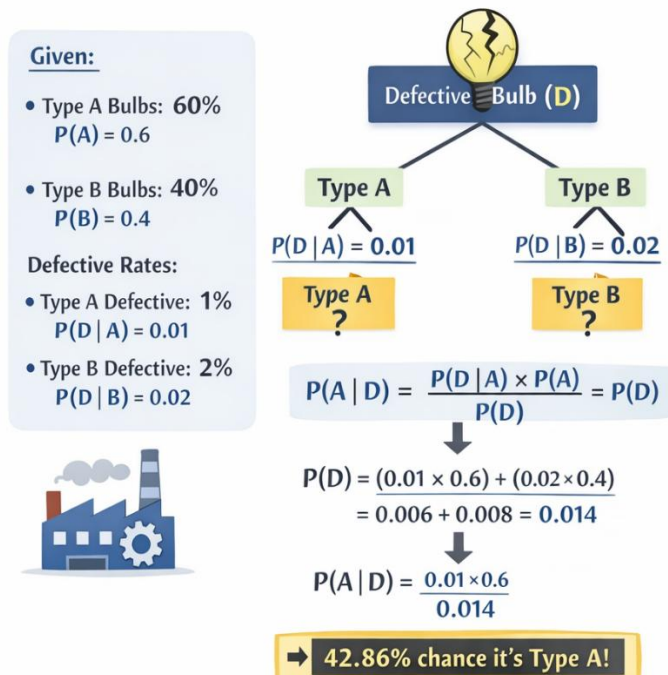
Step 3: Calculate $P(A | D)$

$$P(A | D) = \frac{0.01 \cdot 0.6}{0.014} = \frac{0.006}{0.014} \approx 0.4286$$

So, there's a **42.86% chance** the defective bulb is Type A.

Bayes' Rule Example

If a bulb is defective, what's the probability it is Type A?



Shannongame:

The **Shannon Game** is a simple but powerful experiment introduced by **Claude Shannon** to demonstrate how **context and predictability** work in natural language. It is closely related to the concept of **entropy** in information theory and is often used in NLP to explain language modeling.

In the Shannon Game, a player is given a sentence or text **one character or one word at a time** and is asked to **guess the next character or word**. If the guess is correct, the player continues; if not, they are told the correct symbol and proceed.

How the game works

1. Take a sentence.
2. Hide the next word (or letter).
3. Guess it using context.
4. Count how many wrong guesses you make.
5. Fewer guesses → **less uncertainty** → **lower entropy**.

This game shows that **better prediction is possible when language is more structured and less random**.

The Shannon Game is used to estimate the **entropy of a language**.

- **If a player can accurately predict the next symbol with fewer guesses, the entropy is low.**
- **If many guesses are required, the entropy is high.**

Step-by-step guessing (word level) **The movie was very interesting**

Step 1: Guess first word

Context: *No context (start of sentence)*

- Guess 1: **The** 

Sentence so far:

The

 First word is often predictable in English → **low uncertainty**

Step 2: Guess second word

Context:

The ____

- Guess 1: **movie** ✓

Sentence so far:

The movie

Step 3: Guess third word

Context:

The movie ____

- Guess 1: **is** ✗
- Guess 2: **was** ✓

Sentence so far:

The movie was

➡ Past vs present tense → **some uncertainty**

Step 4: Guess fourth word

Context:

The movie was ____

- Guess 1: **very** ✓

Sentence so far:

The movie was very

Step 5: Guess fifth word

Context:

The movie was very ____

- Guess 1: **good** ✗
- Guess 2: **boring** ✗
- Guess 3: **interesting** ✓

Sentence completed

The movie was very interesting

Guess summary

Word position Word guessed No. of guesses

1	The	1
2	movie	1
3	was	2
4	very	1
5	interesting	3

Total guesses = 8

Original sentence:

The sun rises in the east

Step 1: First word

Context: *Start of sentence*

- Guess 1: **The** ✓

Sentence so far:

The

Step 2: Second word

Context:

The ____

- Guess 1: **sun** ✓

Sentence so far:

The sun

Step 3: Third word

Context:

The sun ____

- Guess 1: **rises** ✓

Sentence so far:

The sun rises

Step 4: Fourth word

Context:

The sun rises ____

- Guess 1: **in** ✓

Sentence so far:

The sun rises in

Step 5: Fifth word

Context:

The sun rises in ____

- Guess 1: **the** ✓

Sentence so far:

The sun rises in the

Step 6: Sixth word

Context:

The sun rises in the ____

- Guess 1: **east** ✓

✓ Sentence completed

The sun rises in the east

Guess summary (correct)

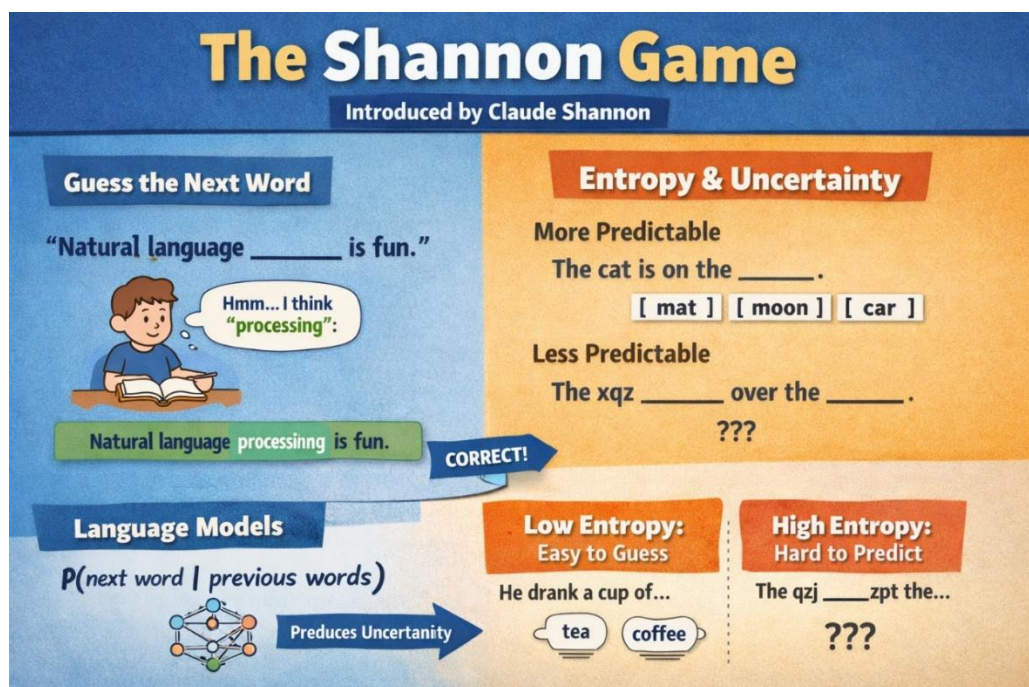
Word	Guesses
The	1
sun	1
rises	1
in	1
the	1
east	1

➡ **Total guesses = 6 (Very Low)**

Applications of Shannon Game:

- Measures entropy (uncertainty) of language
- Evaluates language models
- Helps in text compression
- Used in NLP tasks like prediction and speech recognition
- Basis for auto-complete and predictive typing
- Predictive text & auto-completion (**mobile keyboards, email suggestions**)
- Speech recognition systems (**predicting next words from context**)
- Chatbots & language models (**next-word prediction**)
- Spell check & grammar correction
- Search engines (**query prediction and ranking**)

In essence, the Shannon Game demonstrates that **natural language is highly predictable**, and this predictability can be measured using entropy and modeled using probabilities.



Entropy and Cross Entropy.

Entropy and Cross Entropy are core concepts from information theory and are widely used in NLP and machine learning to measure **uncertainty** and **prediction error**.

Entropy

Entropy measures the **average uncertainty** in a random variable. In simple terms, it tells us **how unpredictable** an outcome is.

Entropy means how difficult it is to guess the next word in a sentence.

- Easy to guess → Low entropy
- Hard to guess → High entropy

The formula for entropy is:

$$H(P) = - \sum_x P(x) \log P(x)$$

Here, $P(x)$ is the probability of outcome x .

Example (Coin toss):

- **Fair coin:** $P(H) = 0.5$,
 $P(T) = 0.5$
 $H = -[0.5 \log 0.5 + 0.5 \log 0.5] = 1 \text{ bit}$

This is **maximum entropy**, because the outcome is most uncertain.

- There are two possible outcomes (like Head or Tail)
- Both outcomes have equal probability = 0.5
- Since both are equally likely, we cannot predict which will occur
- This situation has maximum uncertainty

So the entropy is 1 bit, which is the maximum entropy for two outcomes

- **Biased coin:** $P(H) = 0.9$,
 $P(T) = 0.1$

Entropy formula:

$$H = -\sum p(x) \log_2 p(x)$$

Step 1: Plug in values

$$H = -[0.9 \log_2 0.9 + 0.1 \log_2 0.1]$$

Step 2: Calculate $\log_2$ values

- $\log_2 0.9 \approx -0.152$
- $\log_2 0.1 \approx -3.322$

Step 3: Multiply by probabilities

$$0.9 \times (-0.152) = -0.1368 \quad 0.1 \times (-3.322) = -0.3322$$

Step 4: Sum and apply negative

$$H = -[-0.1368 + (-0.3322)] = -[-0.469] = 0.469 \text{ bits}$$

Biased Coin – Less Uncertainty

- There are two possible outcomes: Head (H) or Tail (T)
- Probabilities: $P(H) = 0.9$, $P(T) = 0.1$
- Head is much more likely than Tail, so the outcome is easier to predict
- This situation has less uncertainty than a fair coin
- Entropy (H) ≈ 0.469 bit $\rightarrow$ lower than maximum

Entropy is low, because the outcome is more predictable.

- **Fair coin $\rightarrow$ max uncertainty $\rightarrow$ entropy = 1**
- **Biased coin $\rightarrow$ less uncertainty $\rightarrow$ entropy < 1**

Cross Entropy

Cross entropy measures **how well one probability distribution predicts another**. It is the average number of bits needed when we use a **predicted distribution** Q to encode data that actually follows the **true distribution** P .

- Cross-entropy measures **how different two probability distributions are**
- It is used when you have:
 1. The **true probability distribution** of outcomes (actual data) $\rightarrow P$
 2. The **predicted probability distribution** from a model $\rightarrow Q$

The formula is:

$$H(P, Q) = - \sum_x P(x) \log Q(x)$$

Here:

- $P(x)$ = true distribution (actual labels)
- $Q(x)$ = predicted distribution (model output)

Step-by-step idea

1. Suppose the **true distribution** says Head = 0.5, Tail = 0.5
 2. Your **model predicts** Head = 0.9, Tail = 0.1
 3. Cross-entropy calculates the **average “surprise”** if you use the predicted distribution Q instead of the true distribution P
- The higher the cross-entropy $\rightarrow$ the **worse the prediction**

Example

True distribution (fair coin):

$$P(H) = 0.5, P(T) = 0.5$$

Predicted by model:

$$Q(H) = 0.9, Q(T) = 0.1$$

Cross-entropy:

$$H(P, Q) = -[0.5 \log_2 0.9 + 0.5 \log_2 0.1] \approx 1.23 \text{ bits}$$

- More than 1 bit $\rightarrow$ model is **less accurate** than perfect prediction (which would be 1 bit)

Example: Predicting Weather

Scenario

- You want to predict **whether it will rain tomorrow**
- True probability (based on historical data):

$$P(Rain) = 0.3, P(No Rain) = 0.7$$

- Your weather model predicts:

$$Q(Rain) = 0.6, Q(No Rain) = 0.4$$

Step 1: Entropy (True uncertainty)

$$H(P) = -[0.3 \log_2 0.3 + 0.7 \log_2 0.7] \approx 0.881 \text{ bits}$$

- Shows **how uncertain the weather really is**

Step 2: Cross-Entropy (Using Model Prediction)

$$H(P, Q) = -[0.3 \log_2 0.6 + 0.7 \log_2 0.4] \approx 1.06 \text{ bits}$$

- Higher than entropy → model **less accurate** → more surprise when outcome happens
 - **Entropy** = actual uncertainty in weather
 - **Cross-entropy** = “extra surprise” using model’s prediction
 - The closer cross-entropy is to entropy → the **better the model**

Applications of Cross-Entropy

1. Machine Learning / Deep Learning
 - Used as a loss function in classification tasks
 - Example: Predicting image labels, spam emails, sentiment analysis
2. Natural Language Processing (NLP)
 - Measures how well a language model predicts the next word
 - Example: Auto-complete, chatbots, translation systems
3. Speech Recognition
 - Evaluates the accuracy of predicted phonemes or words compared to actual speech
4. Weather Prediction / Forecasting
 - Compares predicted probabilities of rain, snow, etc., with actual outcomes
5. Medical Diagnosis
 - Evaluates probabilistic predictions of diseases versus real patient outcomes
6. Recommendation Systems
 - Predicting probabilities of users liking or clicking items

